

Prototype Word

Éditeur de documents collaboratif

Version 1.0 • Durée du projet : 4 semaines

Pourquoi

Problématique

Aujourd'hui, pour rédiger, mettre en forme et collaborer sur des documents, nous dépendons d'outils tiers (Google Docs, Microsoft Word/365, Notion, etc.). Cela pose plusieurs limites : dépendance à des services externes, manque de maîtrise sur les données et le code, et impossibilité d'adapter l'outil à nos besoins spécifiques.

Vision du projet

Construire Prototype Word : une application web permettant de créer, formater et co-éditer des documents en temps réel, sans dépendre d'un éditeur tiers, en maîtrisant de bout en bout l'authentification, le stockage et la collaboration.

Quoi

Objectif principal

Développer en 4 semaines un MVP fonctionnel d'éditeur de texte en ligne avec gestion de comptes, sauvegarde des documents et édition collaborative en temps réel.

Livrables

- Application web déployée et dockerisée (front + back + DB)
- Système d'authentification (création de compte, connexion, JWT)
- Éditeur de texte avec formatage (gras, italique, titres, listes, etc.)
- Sauvegarde et gestion des documents liés à un compte utilisateur
- Collaboration en temps réel sur un même document (Yjs)
- Démo finale fonctionnelle

Périmètre (in-scope)

- Auth simple email/mot de passe avec JWT
- Éditeur de texte riche basique (formatage standard)
- Sauvegarde de documents en base PostgreSQL
- Édition collaborative temps réel via Yjs
- Dockerisation de l'ensemble de l'application

Hors périmètre (out-of-scope)

- Export PDF/Word avancé ou mise en page complexe
- Gestion fine des droits et permissions par document
- Commentaires, suivi des modifications, historique de versions
- Authentification OAuth (Google, Microsoft, etc.)
- Déploiement en production sur un serveur public (hors ngrok pour les tests)

Critères de réussite

- Un compte peut être créé, et l'utilisateur peut se connecter/déconnecter de façon sécurisée
- Un document créé est sauvegardé et reste accessible après reconnexion
- Deux utilisateurs peuvent éditer le même document en simultané et voir les modifications en temps réel
- L'application tourne entièrement via Docker (un seul lancement, sans configuration manuelle)
- La démo finale se déroule sans bug bloquant

Qui

Projet réalisé en petite équipe (binôme/trinôme). Répartition des rôles :

Acteur	Rôle	Responsabilités
Thomas	Backend / Auth	API, base de données, JWT, sécurité
Emanuel, Ewen	Frontend / Éditeur	Interface, éditeur de texte, formatage
Jérémy, Ewen	Collab temps réel / DevOps	Intégration Yjs, Docker, tests, démo
Toute l'équipe	Pilotage	Planning, gestion des bugs

Comment

Organisation

- Travail en sprints hebdomadaires d'une semaine, avec un objectif clair par semaine
- Gestion du code via un dépôt Git partagé (branches par fonctionnalité)
- Suivi des tâches simple (Kanban ou Notion : à faire / en cours / terminé)
- Mise au point chaque matin 10h30
- Groupe chat pour travail à distance

Stack technique envisagée

- Authentification : email/mot de passe, JWT
- Base de données : PostgreSQL via Docker
- Éditeur de texte : librairie d'édition riche (front)
- Collaboration temps réel : Yjs
- Tests entre membres de l'équipe : ngrok pour exposer un environnement local
- Conteneurisation : Docker / Docker Compose pour l'ensemble de l'application

Quand

Jalons et échéances (4 semaines)

Période	Objectifs
Semaine 1	Authentification et gestion de comptes (email/mot de passe, JWT, base de données PostgreSQL via Docker), dockerisation complète de l'application + mise en place du squelette de l'éditeur
Semaine 2	Éditeur complet : formatage du texte, sauvegarde des documents liés à un compte utilisateur
Semaine 3	Collaboration en temps réel avec Yjs, mise en place de ngrok pour les tests entre membres de l'équipe
Semaine 4	Finitions (polish), correction des bugs, préparation de la démo finale

Les risques

Risque	Impact	Mitigation
La collaboration temps réel (Yjs) est plus complexe que prévu	Élevé	Commencer les tests Yjs tôt (dès la semaine 2 en parallèle), prévoir un fallback simple (sauvegarde périodique) si besoin
Problèmes de dockerisation en fin de projet	Moyen	Dockeriser progressivement dès la semaine 1, ne pas tout reporter à la semaine 4
Manque de temps / périmètre trop ambitieux	Élevé	Respecter strictement le hors-périmètre défini, prioriser les critères de réussite
Disponibilité inégale des membres de l'équipe	Moyen	Répartition claire des rôles